

Open TabFM-Auto: What Survives When the Frozen Model, the LLM and the Agent Are All Open?

Yarin Benizri

Hebrew University of Jerusalem · experiments run on a Slurm cluster with H200 GPUs · code: open-tabfm-auto, frozen-embeddings-retrieval · working report, 3 October 2026

ABSTRACT. TabFM-Auto (Fu et al., 2026) showed that a coding agent which evolves the data pipeline around a *frozen* tabular foundation model, judged by cross-validation, reaches the top of TabArena. The published result rests on a closed 400M-parameter model and on frontier LLM agents. We re-implement the method with open components only and vary one axis at a time: the frozen backbone (TabPFN v2/2.5, TabICLv2, Kumo Tabular S/M/L, EXAONE), the LLM (Qwen3-Coder-30B, Qwen3-32B, GLM-4.5-Air, gpt-oss-20b, served locally with vLLM), the agent harness (a minimal tool loop, and the free CLI agents pi, aider and Qwen Code), the judge (single vs repeated cross-validation) and the final pick (greedy, gated, ensemble). On the 17 small TabArena datasets, scored on all 30 official splits, no open setup reproduces the paper's mean gain of +4.6%: the best is a greedy search with no LLM at +0.5%, and the two open-LLM agents average below zero. The strongest open backbone, Kumo Tabular-L, already matches the paper's frozen TabFM with no search at all (better on 10 of 17 datasets), and the search then hurts it. Per-split analysis shows why: a candidate chosen by cross-validation on one split improves only about half of the other splits of the same data, and a larger budget selects more of that noise. The harness matters more than the model where an entity structure is present: the same 30B model gains 50% through pi and 0% through a minimal tool loop. As a side study we test frozen hidden states as training-free entity embeddings on RelBench rel-hm and find them no better than their input features for recommendation and marginally better (relational model only) for a customer-level label. We release the code, the per-split results and five reproducible figures.

1. Introduction

The paper's claim is a mechanism: keep the predictor frozen, let an LLM turn column names and task text into code around it, and judge every candidate with a cheap cross-validation (CV) signal. Two things make the result hard to reuse: the frozen model is not released, and the search spends thousands of frontier-LLM calls per dataset. We separate the mechanism from those ingredients and measure what each is worth, keeping the pipeline contract, the judge and the evaluation protocol identical to the paper.

Contributions. (i) An open, MIT-licensed implementation of the full loop with pluggable backbones, LLMs and harnesses, plus a SQL mode. (ii) A backbone study across seven open tabular foundation models (TFMs). (iii) An LLM and harness study including a no-LLM baseline. (iv) A diagnosis of why gains do not reproduce at small scale, with two mitigations (a repeated judge and an acceptance slice). (v) A retrieval study of frozen hidden states as entity embeddings on a synthetic database and on RelBench.

2. Method and setup

Contract. A pipeline is a Python file with four hooks (preprocess, engineer ≤ 500 columns, sample

context views, postprocess) and constructor settings for the frozen model. The identity pipeline P0 passes the raw columns through. The judge is 3-fold CV on the training split, run in a sandbox the agent calls as tabfm-eval; the held-out split never enters the workspace. The best-CV candidate P* is frozen and scored once.

Protocol. TabArena v0.1, the 17 datasets with $\leq 2,500$ rows; search on the training part of the first official split, then P0 and P* scored on all 30 official splits; the paper's per-dataset numbers (its Table 9) are vendored for direct comparison. *Gain* is the relative error reduction of P* over P0. Budgets are 16–24 evaluations per dataset (the paper: about 100).

Components. Backbones from Hugging Face; LLMs served by vLLM on one H200 each; harnesses: Claude Code (reference, local only), an OpenAI-compatible five-tool loop, free CLI agents (pi, aider, Qwen Code), and a greedy coordinate search over generic operations with no LLM.

3. Results

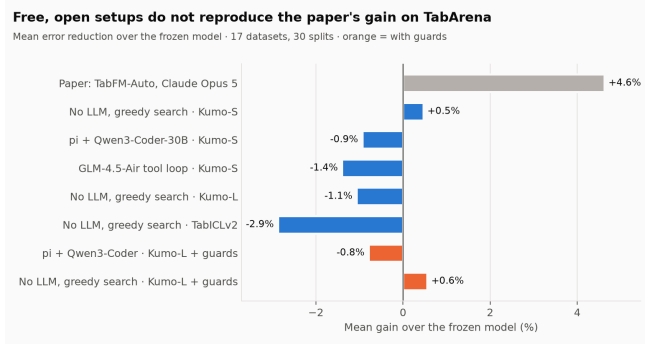


Figure 1. Mean gain of the searched pipeline over the frozen model alone on the 17 small TabArena datasets, all 30 official splits. Orange: with the guards (repeated judge on small tables, gated pick, acceptance slice). No free setup reproduces the paper; the guarded no-LLM search on Kumo-L is the only Kumo-L setup above zero.

No open setup reproduces the paper's gain (Fig. 1). The greedy no-LLM search around Kumo Tabular-S gains +0.5% on average (9 of 17 datasets improve); pi + Qwen3-Coder-30B averages -0.9% and the GLM-4.5-Air tool loop -1.4%. The paper reports +4.6% with Claude Opus on the same datasets. Where the paper gains most (anneal +17.6%, Marketing_Campaign +15.8%, airfoil +14.6%) the open LLM agents lose (pi: -5.2%, -14.2%, +3.7%).

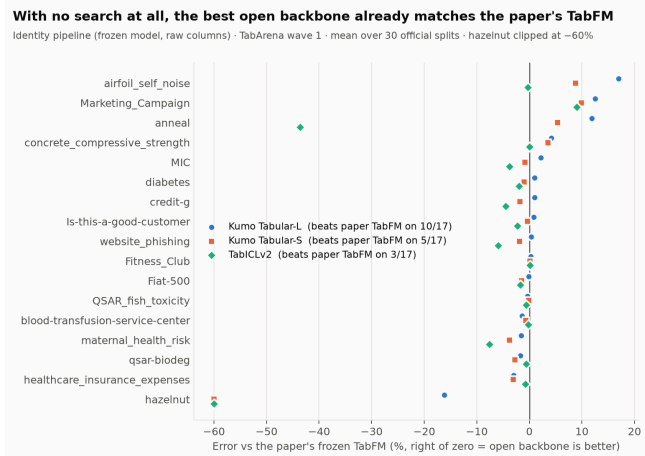


Figure 2. Error of each open backbone with the identity pipeline, relative to the paper's frozen TabFM, per dataset. Kumo Tabular-L is better on 10 of 17 with no search.

The best open backbone needs no search (Fig. 2). Kumo Tabular-L's identity pipeline beats the paper's TabFM on 10 of 17 datasets (mean +1.6% relative) and the paper's *searched* TabFM-Auto on 4; the same greedy search then lowers its average by 1.1%. TabICLv2 is 10.9% behind TabFM and loses 2.9% more under search. The stronger the backbone, the less a generic feature search can add, and the more a noisy judge costs.

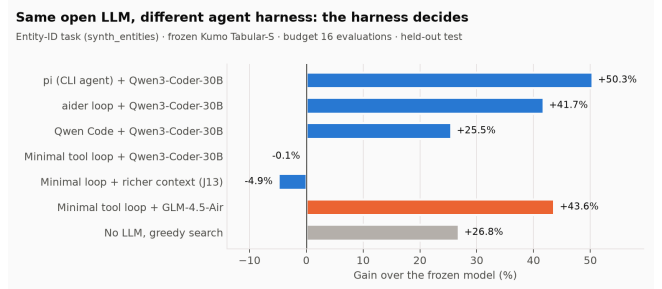


Figure 3. Entity-ID task, same frozen model, 16 evaluations. The agent harness decides what a fixed open LLM finds.

The harness matters more than the LLM (Fig. 3). On a synthetic task whose description names an entity structure, Qwen3-Coder-30B cuts the error by 50% inside pi, 42% inside an alder loop and 26% inside Qwen Code, and by about 0% inside our minimal tool loop; giving the minimal loop the same starting context (file listing, pipeline, data summary) does not close the gap (-4.9%), so the difference is in how the agent iterates. GLM-4.5-Air finds the same frequency encodings even in the minimal loop (+44%). All three CLI agents converge on the same role×resource counts. These gains do not carry to TabArena, where no dataset text names such a structure.

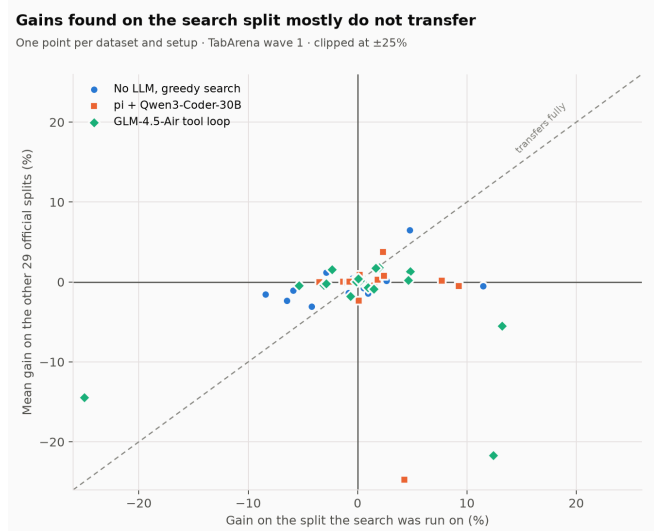


Figure 4. Gain on the split the search used vs the mean gain on the other 29 official splits, one point per dataset and setup. Points far from the diagonal are candidates that looked good once.

Why: the search selects CV noise (Fig. 4). A candidate chosen by 3-fold CV on one split improves only 41-57% of the other splits of the same data, a coin flip, for every setup. With 500-2,500 rows a CV difference of one or two percent is inside the noise. Raising the budget to 64 evaluations on the five datasets where the paper gains most makes every setup worse (heuristic +2.9% → +1.1%, pi -2.9% → -4.3%, GLM +0.1% → -8.4%), and the LLM agents stop on their own

after 8–22 evaluations. The backbone itself is nondeterministic ($\pm 1\%$ RMSE, $\pm 30\%$ small-fold log-loss on repeated calls), a noise term now removed by seeding.

Mitigations. A repeated 3 $\times$ 3-fold judge on tables under 1,000 rows moves the heuristic from +1.4% to +2.1% and pi from -1.4% to -0.5% on those tables, and shrinks pi's loss on a 569-row table from -17% to -5% . Conservative final picks (replace P0 only if the paired per-fold improvement exceeds 1 or 2 standard errors; or

average the three best candidates), re-scored on all splits, change the heuristic's mean gain from +0.25% to +0.49% / +0.29% / +0.25% and cut the datasets losing more than 1% from 6 to 3 at the 2-s.e. gate, but every LLM family keeps at least one dataset whose candidate is confidently better in CV and 10–15% worse everywhere else. CV-based rules cannot catch those; an acceptance slice of the training split that the pick must beat P0 on (implemented, under evaluation) can.

Table 1. Open LLMs at a fixed harness (our tool loop), frozen Kumo Tabular-S: held-out gain on the three bundled tasks, on three TabArena-Lite datasets (airfoil / blood-transfusion / credit-g), and wall time per search on one H200.

LLM	physics	entities	breast_cancer	airfoil	blood	credit-g	min / search
GLM-4.5-Air-FP8 (106B-A12B)	+5.1%	+43.6%	-38.9%	-0.9%	+1.4%	+1.4%	5
gpt-oss-20b	+4.1%	-8.9%	-7.9%	+2.0%	-1.1%	-0.2%	5
Qwen3-Coder-30B-A3B (two runs)	+1.3 / +2.9%	-1.1 / +1.0%	-2.9 / -57%	-0.9 / +2.3%	+2.5 / -0.9%	-0.3 / -0.4%	1.5
Qwen3-32B (reasoning)	-0.2%	-3.8%	-2.9%	+0.7%	+0.6%	+0.3%	17–30
<i>no LLM, greedy search</i>	+0.6%	+25–29%	0%	—	—	—	~10

Every open setup overfits the smallest table (569 rows): the CV-best candidate is worse on test by 3% to 57% under a single 3-fold judge.

4. Frozen hidden states as entity embeddings

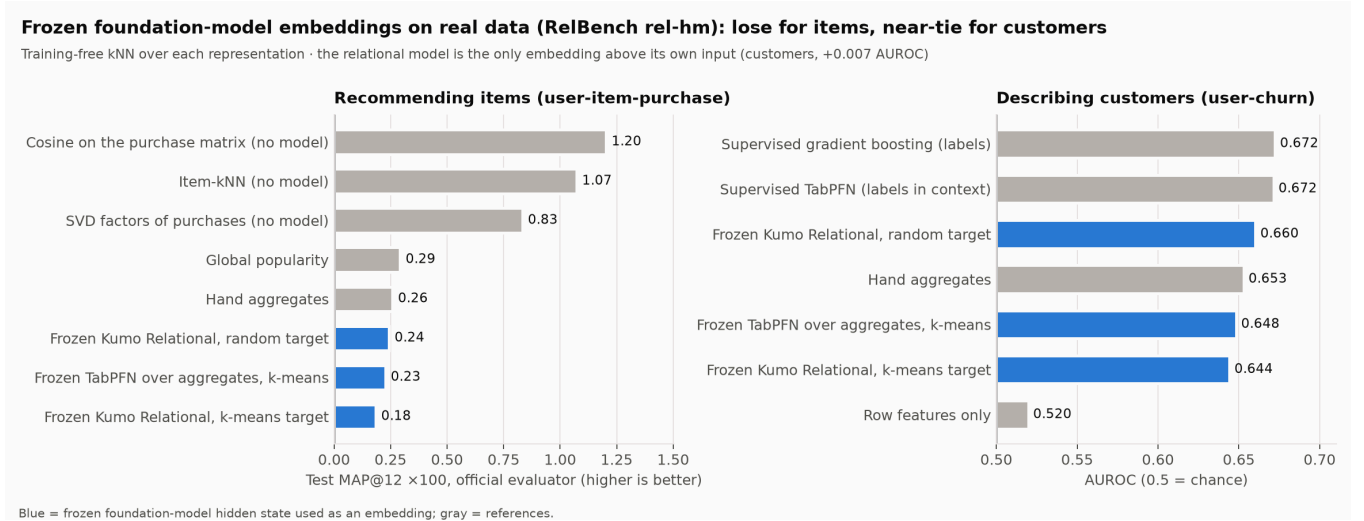


Figure 5. RelBench rel-hm. Left: user-item-purchase, test MAP@12 ($\times 100$) with the official evaluator, training-free kNN over each customer representation. Right: user-churn, kNN-probe AUROC over 20,000 customers. Blue: frozen foundation-model hidden state used as the embedding.

Synthetic database. On a shop database with planted customer segments (chance P@10 0.17), the frozen TabPFN hidden state over hand aggregates with k-means pseudo-labels as the in-context target scores 0.82 (a GNN trained on the database: 0.88; OpenRFM 0.25; Kumo Relational 0.62 with the k-means target, 0.33 with a random one). The in-context target decides what the embedding keeps.

Real data (Fig. 5). For recommending items, every frozen embedding (0.18–0.24) is below the

hand aggregates (0.26) and far below plain cosine similarity on the purchase matrix (1.20); feeding SVD factors of that matrix through the frozen model halves their value ($0.83 \rightarrow 0.42$). For a customer-level label, the frozen TabPFN equals its input (0.648 vs 0.653) and the frozen relational model with a random target is the single row above its input (0.660), still below a supervised model on the same features (0.673). Frozen FM hidden states are entity-level summaries of the features they are given, not

retrieval embeddings; the interaction matrix itself is the right representation for recommendation.

5. Recommendation and limitations

End-to-end test of the guards (J17). With Kumo Tabular-L, a repeated judge on small tables, a 1-s.e. gated pick and a 20% acceptance slice, the no-LLM search goes from -1.07% (6 datasets losing more than 1%) to **+0.56%** (1 dataset), and its pipeline beats the paper's TabFM-Auto on 5 of 17 datasets. The guards mostly refuse changes (median gain 0.0%); anneal moves from -8.8% to $+5.4\%$. The same guards around pi + Qwen3-Coder give -0.79% , with one failure that passes every check (Marketing_Campaign -14.7%).

Recommendation. For a free, open setup on small tables: use Kumo Tabular-L with the identity pipeline; add the guarded no-LLM search (`--cv-repeats auto --select gated1 --accept-holdout 0.2`) when a small, reliable gain is worth an hour of GPU; use an LLM agent (pi with Qwen3-Coder-30B, or GLM-4.5-Air in a tool loop) only when the task text names an entity

structure. Limitations: budgets $4-6\times$ below the paper's; the retrieval study covers one relational benchmark plus one public row-embedding benchmark (TEmBed, text-heavy, where TabPFN is near chance whatever its context target); the backbone's nondeterminism adds noise to comparisons made before seeding was introduced.

References

- Fu, Z. et al. (2026). TabFM-Auto: Self-Evolving Pipelines for Tabular Foundation Models. arXiv:2609.37989.
- Erickson, N. et al. (2025). TabArena: A Living Benchmark for Machine Learning on Tabular Data.
- Robinson, J. et al. (2024). RelBench: A Benchmark for Deep Learning on Relational Databases.
- Hollmann, N. et al. (2025). Accurate predictions on small data with a tabular foundation model (TabPFN). Nature.
- Qu, J. et al. (2025). TabICL: A Tabular Foundation Model for In-Context Learning on Large Data.
- NVIDIA (2026). Structured Data Models: Kumo Tabular and Kumo Relational (open weights).
- Experiments and this report were produced with Claude Code; every number is read from the run directories by `paper/fill_results.py` and `paper/make_figures.py`.